COMP7940 Cloud Computing
Lab 6: Chatbot in Docker

Name: ZHAO Zhan
Student ID: 25482351
GitHub: @Extious

March 29, 2026

1. Docker Image and Container Workflow

The chatbot was containerized using a `Dockerfile` based on `python:3.12-slim`. Dependencies are installed from `requirements.txt`, and only the Python modules `chatbot.py` and `ChatGPT_HKBU.py` are copied into the image. Runtime credentials are *not* baked into the image; instead, `config.ini` is supplied at container startup by bind-mounting a host file into `/comp7940-lab/config.ini`. The image was built with:

```
docker build -t lab6-chatbot .
```

The container was started in detached mode with a volume mount so the bot reads the local configuration file inside the container:

```
docker run -d --name chatbot-container \
  -v /absolute/path/to/config.ini:/comp7940-lab/config.ini \
  lab6-chatbot
```

The lab instructions use the image name `chatbot`; the same workflow applies after tagging the image as `chatbot` with `docker build -t chatbot .`.

2. Container Logs

Figure 1 shows `docker logs chatbot-container` output after a successful startup: configuration is loaded, the Telegram application is initialized, message handling is registered, and polling begins.

3. Write-Up Questions

Q1: Explain why a Docker image should not contain `config.ini`

Including `config.ini` in the image is discouraged when that file holds secrets (Telegram bot token, API keys, endpoints) or environment-specific settings:

1. **Secrets become part of image layers.** Anything `COPY`ed into an image is stored in the image filesystem and its layers. Anyone who can `docker pull` the image, receive a `.tar` export, or access a registry can extract files from those layers and recover the credentials [1].

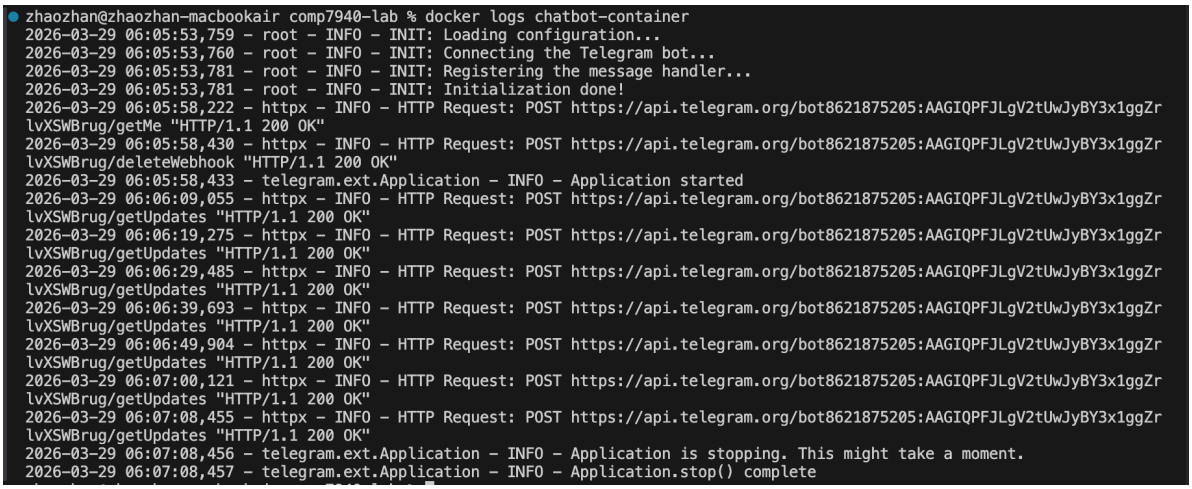

Figure 1: Docker logs for `chatbot-container`, showing initialization and Telegram polling.

2. **History and sharing.** Image layers persist in build cache and registries; accidentally pushing an image that embeds secrets leaks them broadly and permanently unless layers are fully discarded. Re-tagging or rebuilding does not remove secrets already published in an old layer.

3. **Separation of code and configuration.** The same image should be able to run in development, staging, or production with different tokens and URLs. Twelve-factor style guidance treats config as environment-specific data supplied at runtime (files, environment variables, or secret stores), not baked into the build artifact [2].

4. **Operational rotation.** If a token is rotated, mounting `config.ini` or injecting env vars avoids rebuilding and redeploying the image solely to change secrets; only the runtime configuration or orchestration definition needs updating.

Therefore the `Dockerfile` copies only application code and dependencies. The file `config.ini` is mounted when the container runs.

Q2: Provide a tar file of the chatbot Docker image

The chatbot image was exported to a single archive using:

```
docker save -o lab6-chatbot.tar lab6-chatbot:latest
```

The resulting `lab6-chatbot.tar` file is submitted alongside this report (equivalent to the lab's `docker save -o chatbot.tar chatbot` after building/tagging the image as `chatbot`). On another machine, the image can be restored with `docker load -i lab6-chatbot.tar`, then run with the same bind mount for `config.ini` as in Section 1.

References

[1] Docker Inc. Docker documentation. Available at `https://docs.docker.com/`, 2026. Accessed: 2026-03-29.

[2] Heroku. The twelve-factor app — config. Available at `https://12factor.net/config`, 2017. Accessed: 2026-03-29.